CloudShield AI / DAIL

14 — Development & DevOps Specification

Version 1.0 | Final Development/DevOps Baseline | 1 September 2026

Purpose: Define the engineering workflow, repository structure, local development environment, configuration management, infrastructure-as-code workflow, CI/CD, testing gates, deployment model, environment separation, secrets handling, versioning, observability, rollback, operational safety, and developer Definition of Done required to implement DAIL consistently with Specifications 01–13.

Document Control

Field	Value
Document ID	CSAI-DAIL-DEVOPS-014
Status	FINAL DEVELOPMENT/DEVOPS BASELINE
Parent documents	01 SRS; 02 Architecture; 03 Functional; 04 Terraform/AWS; 05 Domain/Data; 06 Lifecycle; 07 Identity/Dependency; 08 Impact/Invalidation; 09 Verification/Promotion; 10 LLM Integration; 11 Evidence/Logging; 12 Testing/QA; 13 Experiment Harness
Primary outputs	Source Repository, CI/CD Pipeline, Environments, Infrastructure, Release Artifacts
Engineering principle	Reproducible builds, fail-closed safety, versioned infrastructure, automated quality gates.
Deployment principle	No deployment or infrastructure mutation bypasses controlled automation.

1. Scope

This specification defines how DAIL is developed, tested, packaged, deployed, operated, and maintained.

- Repository architecture.
- Branching and pull requests.
- Local setup.
- Language/runtime/toolchain policy.
- Configuration management.
- Terraform workflow.
- AWS environment separation.
- CI/CD.
- Build and packaging.
- Testing gates.
- Database migrations.
- Secrets/configuration.
- Observability.
- Release/versioning.
- Rollback/recovery.
- Developer workflow.

- Operational security.

2. Engineering Principles

- Keep deterministic safety logic separate from probabilistic LLM integration.
- Treat infrastructure as code.
- Never commit secrets.
- Every change is reviewed and tested before merge.
- CI is the first enforcement layer; production controls are the second.
- Safety-critical code requires negative/adversarial tests.
- Trusted State changes only through the application promotion transaction.
- Environment differences are explicit configuration, not hidden code paths.
- Version every semantic contract that affects reproducibility.

3. Logical Repository Structure

```
cloudspartanx/
├── apps/
│   ├── api/
│   ├── worker/
│   └── ui/
├── core/
│   ├── domain/
│   ├── identity/
│   ├── dependency/
│   ├── impact/
│   ├── verification/
│   └── promotion/
├── llm/
│   ├── adapter/
│   ├── prompts/
│   ├── schemas/
│   └── validation/
├── evidence/
├── experiments/
├── fixtures/
├── infrastructure/
│   └── terraform/
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── contract/
│   ├── adversarial/
│   └── e2e/
├── docs/
├── scripts/
└── .github/
    └── workflows/
```

Exact module names may evolve during implementation, but the architectural separation between domain logic, infrastructure, LLM integration, evidence, experiments, and tests must remain.

4. Component Ownership

Area	Primary responsibility
apps/api	API/orchestration entry points.
apps/worker	Asynchronous trial/analysis execution.
core/domain	State, invariant, candidate, lifecycle semantics.
core/identity	Identity/change analysis.
core/dependency	Dependency graph construction.
core/impact	Impact/invalidation.
core/verification	Deterministic verification.
core/promotion	Promotion decision/transaction.
llm	Model interaction and patch generation.
evidence	Evidence/audit persistence.
experiments	Experiment execution/metrics.
infrastructure	AWS/Terraform definitions.
tests	Automated quality system.

5. Dependency Direction

```
UI/API
  ↓
Application Services
  ↓
Domain Core
  ├── Identity
  ├── Dependency
  ├── Impact
  ├── Verification
  └── Promotion

Adapters:
  ├── LLM
  ├── AWS
  ├── Database
  └── Artifact Storage

Infrastructure must not leak into pure domain rules.
```

Domain algorithms should be testable without AWS access.

6. Technology Baseline

Layer	Baseline
Language	Python 3.x for initial implementation.
Infrastructure	Terraform.
Cloud	AWS.
Database	Relational persistence as defined by implementation architecture.
Artifact storage	Object storage/content-addressed artifacts.
API	Typed HTTP API.
LLM	Provider-neutral adapter.
Testing	Pytest-style automated test stack.
CI	Git-based CI workflow.

Containerization	Container-based deployment where selected by architecture.

Exact minor versions and provider/model versions must be pinned in repository configuration before implementation begins.

7. Version Pinning

- Pin language/runtime version.
- Pin package dependencies.
- Pin Terraform version.
- Pin AWS provider version.
- Pin container base image digest where practical.
- Pin schema versions.
- Pin verifier/policy versions.
- Pin experiment fixture versions.
- Record model/deployment versions.

8. Repository Initialization

1. Create repository and protected default branch.
2. Add project license/metadata as required.
3. Add dependency lockfile.
4. Add formatting/lint/static-analysis configuration.
5. Add test configuration.
6. Add pre-commit/developer checks where useful.
7. Add CI workflow.
8. Add Terraform directory and environment structure.
9. Add documentation directory.
10. Add initial architectural decision records where needed.

9. Branching Strategy

Branch	Purpose
main	Protected, releasable baseline.
feature/*	Normal implementation work.
fix/*	Defect fixes.
experiment/*	Research/harness changes when isolated.
release/*	Optional release preparation.

Direct pushes to main are prohibited except controlled administrative recovery.

10. Pull Request Requirements

- Linked requirement/specification section.
- Description of semantic change.
- Tests added/updated.
- Security impact assessment where relevant.
- Evidence/lifecycle impact assessment for safety-critical changes.
- CI green.

- Reviewer approval.
- No secrets or generated garbage.
- Documentation updated when contracts change.

11. Code Ownership

Safety-critical modules should have explicit reviewers/owners.

Module	Required review focus
Identity	Correspondence correctness and uncertainty.
Dependency	Edge semantics and false-negative risk.
Impact	Invalidation completeness.
Verification	Predicate correctness and evidence.
Promotion	Fail-closed transaction semantics.
LLM adapter	Isolation, parsing, secret handling.
Evidence	Integrity, lineage, retention.
Experiment harness	Metric validity/reproducibility.

12. Local Development

- Provide one documented bootstrap command.
- Provide deterministic test command.
- Provide formatting/lint command.
- Provide local database/storage setup.
- Provide local fixture runner.
- Provide local LLM mock/stub.
- Provide local experiment runner.
- Avoid requiring AWS credentials for core tests.

13. Local Environment

```
Developer machine
  │
  ├── application
  ├── local database
  ├── local artifact store/mock
  ├── fixture set
  └── mocked LLM provider

Optional:
  AWS sandbox
```

The default developer workflow should be able to execute core DAIL tests without mutating cloud infrastructure.

14. Configuration Model

```
config/
  application
  database
  artifact_store
```

```
llm
verification
promotion
experiments
observability
```

- Environment variables for deployment-specific values.
- Version-controlled defaults for non-secret configuration.
- Explicit schema validation at startup.
- No hidden environment-dependent behavior.

15. Configuration Precedence

Configuration precedence must be deterministic.

```
1. Built-in safe defaults
2. Versioned configuration file
3. Environment-specific configuration
4. Environment variables / secret references
5. Explicit runtime override only where approved
```

Unsafe configuration must fail startup rather than silently default to permissive behavior.

16. Secrets Management

- Use a managed secret mechanism in AWS environments.
- Do not commit secrets to Git.
- Do not place secrets in Terraform variables committed to source.
- Do not log secrets.
- Use least-privilege IAM permissions.
- Rotate credentials according to deployment policy.
- Use local mock credentials for tests where possible.

17. AWS Environment Separation

Environment	Purpose
dev	Developer integration and manual testing.
test	Automated integration/sandbox validation.
experiment	Controlled research execution.
staging	Release candidate validation.
production	Only if/when production deployment is required by the project scope.

State and credentials must not be unintentionally shared between environments.

18. Terraform Repository Layout

```
infrastructure/terraform/
├── modules/
├── environments/
```

```
│   ├── dev/
│   ├── test/
│   ├── experiment/
│   └── staging/
├── policies/
└── versions.tf
```

Reusable modules contain infrastructure semantics; environment roots compose modules and provide environment-specific configuration.

19. Terraform Workflow

11. terraform fmt
12. terraform validate
13. static/security validation
14. terraform init with locked provider versions
15. terraform plan
16. review plan
17. apply only through authorized workflow
18. record deployment metadata

Production or shared-environment applies should require CI/CD approval and a reviewed plan.

20. Terraform Safety Rules

- Remote state must be protected.
- State locking must be enabled where supported.
- Separate environments/states.
- Least-privilege deployment role.
- No credentials in state unless unavoidable and protected.
- Review destructive plan actions.
- Do not run ad-hoc production applies from developer laptops.

21. Infrastructure State

Terraform state is infrastructure state and is distinct from DAIL Trusted State. The two must never be conflated.

```
Terraform State
  = cloud resource provisioning truth

DAIL Trusted State
  = trusted semantic infrastructure candidate baseline
```

22. CI Pipeline

```
Pull Request
  ↓
format/lint
  ↓
```

```
static analysis
  ↓
unit tests
  ↓
contract tests
  ↓
integration tests
  ↓
adversarial/security tests
  ↓
build
  ↓
Terraform validation
  ↓
PR approval
  ↓
merge
```

23. CI Required Gates

Gate	Requirement
Formatting	PASS.
Lint	PASS.
Static analysis	PASS.
Unit tests	PASS.
Contract tests	PASS.
Safety regression tests	PASS.
Adversarial tests	PASS.
Build/package	PASS.
Terraform validate	PASS.
Dependency/security scan	PASS under configured policy.

24. CD Pipeline

```
main
  ↓
build immutable artifact
  ↓
test/revalidate
  ↓
deploy dev/test
  ↓
integration/smoke
  ↓
staging
  ↓
approval
  ↓
release
```

Production deployment, if required, must add explicit approval and environment protection.

25. Artifact Build

- Build from a pinned commit.
- Embed application version/commit metadata.
- Generate immutable artifact identifier.
- Run tests before publishing.
- Publish only after CI gates pass.
- Do not mutate published artifacts.

26. Container Requirements

- Minimal base image.
- Pinned dependencies.
- Non-root process where practical.
- No build secrets embedded in image.
- Health check.
- Deterministic startup configuration.
- Image scanning in CI.
- Immutable image tag/digest for deployment.

27. Database Migration Workflow

19. Create migration in source control.
20. Run schema validation/tests.
21. Apply to isolated test environment.
22. Validate backward compatibility where required.
23. Deploy application compatible with schema.
24. Record migration version.

A migration must not silently destroy evidence, Trusted State lineage, or experiment records.

28. Backward Compatibility

APIs, evidence schemas, and persisted records require explicit compatibility decisions.

- Version breaking API contracts.
- Version evidence schemas.
- Support migration paths for stored records.
- Never reinterpret historical evidence using a newer incompatible schema without explicit migration.

29. Observability

- Application logs.
- Audit events.
- Evidence references.
- Metrics.
- Health checks.
- Dependency/provider status.

- LLM request metrics.
- Experiment metrics.
- Deployment metadata.

30. Health Checks

Check	Purpose
Liveness	Process is running.
Readiness	Required dependencies/configuration are usable.
Database	Connectivity/schema compatibility.
Artifact store	Required storage access.
LLM provider	Optional dependency health; must not affect startup if generation is intentionally disabled.

Health checks must not expose secrets or sensitive infrastructure details.

31. Monitoring

- Error rate.
- Latency.
- Queue depth if asynchronous.
- Verification failure rate.
- Promotion/rejection rate.
- LLM provider failures.
- Evidence persistence failures.
- Infrastructure deployment failures.
- Experiment run failures.

32. Alerting

Alerts should prioritize conditions that can compromise safety or availability.

- Unexpected Trusted State transition failure.
- Evidence integrity failure.
- Promotion transaction errors.
- Repeated verifier errors.
- Secret leakage detection.
- Database corruption/migration failure.
- Repeated deployment failure.
- Critical dependency outage.

33. Release Versioning

Use a semantic project version for released software and independent versions for domain contracts where required.

```
release_version
commit_sha
schema_versions
policy_versions
verifier_versions
```

```
experiment_protocol_versions
```

A release artifact must identify the code/configuration versions that produced it.

34. Release Process

25. Freeze release candidate commit.
26. Run full CI suite.
27. Generate release metadata.
28. Build immutable artifact.
29. Deploy to staging.
30. Run smoke/integration suite.
31. Review evidence/QA report.
32. Approve release.
33. Deploy through protected pipeline.
34. Record deployment event.

35. Rollback Strategy

- Application rollback uses previous immutable artifact.
- Infrastructure rollback uses reviewed Terraform plan/state procedure.
- Database rollback must follow migration-specific policy; prefer forward-compatible migrations.
- DAIL Trusted State is not rolled back by deployment rollback automatically.
- If a bad Trusted State was promoted, remediation follows the DAIL lifecycle rather than silently rewriting history.

36. Incident Handling

35. Detect and classify incident.
36. Protect Trusted State and evidence.
37. Stop unsafe automation if necessary.
38. Preserve logs/artifacts.
39. Identify affected versions/states.
40. Apply controlled mitigation.
41. Restore service.
42. Add regression test.
43. Document root cause.

37. Emergency Stop

The system must support disabling candidate generation or promotion without deleting historical state.

- Disable LLM generation.
- Disable automatic promotion.
- Keep read/audit capabilities available where possible.
- Record configuration change as an audit event.
- Require explicit re-enable after validation.

38. Feature Flags

Feature flags may control non-semantic rollout behavior but must not create hidden safety-policy variants.

- Version safety-relevant flags.
- Record active flags in experiment/deployment metadata.
- Default safety-critical flags to the conservative setting.
- Do not use feature flags to bypass required verification.

39. Dependency Management

- Lock direct dependencies.
- Review transitive security advisories.
- Avoid unnecessary packages.
- Pin major/minor versions according to project policy.
- Record dependency update changes.
- Run tests after updates.

40. Static Analysis

- Type checking where applicable.
- Linting.
- Security scanning.
- Dependency vulnerability scanning.
- Terraform static analysis.
- Secret scanning.
- Dead-code/unused dependency checks where practical.

41. Code Quality Rules

- Small cohesive modules.
- Explicit types for core domain contracts.
- Pure functions for deterministic algorithms where practical.
- No hidden global state.
- No direct provider calls from domain logic.
- Explicit error handling.
- No silent exception swallowing.
- No duplicated safety policy.

42. Logging Development Rules

- Use structured logs.
- Pass correlation IDs.
- Never log secrets.
- Use stable event names.
- Do not make logs the source of truth for lifecycle state.
- Use evidence/audit records for material decisions.

43. Testing in Development

Every implementation change should run the narrowest relevant tests first, followed by the broader suite before merge.

```
changed module
  ↓
targeted unit tests
  ↓
contract/integration tests
  ↓
safety regression suite
  ↓
full CI
```

44. Local Experiment Workflow

- Select immutable experiment version.
- Run fixture validation.
- Run local/mock condition.
- Persist trial artifacts.
- Run metric extraction.
- Inspect QA report.
- Promote to AWS sandbox only when required.

45. Environment Promotion

```
Code:
  dev → test → staging → production

Experiment:
  fixture validation → controlled experiment environment

Infrastructure:
  plan → review → approved apply
```

Promotion of software artifacts and promotion of DAIL Trusted State are separate concepts and must remain separate.

46. Deployment Safety

- Deployments cannot directly modify DAIL Trusted State.
- Deployment identity must have only required AWS permissions.
- Shared environment changes require review.
- Terraform plan must be retained for audited environments.
- Deployment metadata must be linked to application version.

47. Operational Data Protection

- Encrypt data in transit.
- Encrypt sensitive stored data according to AWS/storage policy.
- Restrict database access.
- Restrict raw LLM artifact access.
- Protect Terraform state.
- Use IAM roles rather than long-lived access keys where possible.

48. Backup and Recovery

- Back up authoritative application data according to environment policy.
- Protect Trusted State lineage.
- Protect evidence artifacts.
- Test restore procedures.
- Record recovery events.
- Do not restore data into production without validation.

49. Recovery Invariants

After recovery, the system must preserve:

- Trusted State ordering.
- Evidence lineage.
- Candidate history.
- Promotion decision history.
- Experiment trial history.
- Artifact hashes.
- Schema/version metadata.

50. CI/CD Security Boundaries

- CI credentials use least privilege.
- Pull requests from untrusted forks must not receive privileged secrets.
- Production deployment credentials are environment-protected.
- Terraform applies require controlled identity.
- Secrets are injected at runtime, not source-controlled.

51. Dependency/Provider Failures

Failure	Behavior
LLM unavailable	Generation attempt fails/retries; no unsafe promotion.
AWS API unavailable	Operation records failure; no false PASS.
Database unavailable	Fail operation safely; preserve previous trusted state.
Artifact store unavailable	Do not finalize decision if required evidence cannot persist.
Terraform provider failure	Deployment fails; no assumption of successful apply.

52. Development Documentation

- README/bootstrap guide.
- Architecture overview.
- Local development guide.
- Testing guide.
- Terraform/deployment guide.
- Experiment guide.
- Incident/recovery guide.
- Environment configuration reference.
- Release procedure.

53. Change Classification

Change	Minimum process
Documentation-only	Review + CI as configured.
Non-safety application	Review + relevant tests.
Safety-critical logic	Review + unit + adversarial + integration + regression tests.
Invariant definition	Impact/verification/test/experiment review.
Promotion logic	Full lifecycle + transaction/concurrency tests.
Infrastructure IAM/network	Terraform plan + security review + integration validation.
LLM prompt/model configuration	Version + experiment impact assessment.

54. Definition of Done — Normal Feature

- Implementation complete.
- Tests added/updated.
- Lint/static checks pass.
- Documentation updated.
- CI green.
- No secrets.
- Review approved.
- Deployment path validated where applicable.

55. Definition of Done — Safety-Critical Feature

- Requirement mapped.
- Threat/failure analysis completed.
- Positive and negative tests.
- Adversarial tests.
- Regression test.
- Evidence/audit assertions.
- Concurrency/failure behavior tested where relevant.
- Full CI green.
- Reviewer with ownership approval.
- Versioning/migration impact assessed.

56. Definition of Done — Infrastructure Change

- Terraform fmt/validate pass.
- Plan reviewed.
- Security impact assessed.
- No unexpected destructive actions.
- Environment target explicit.
- Deployment identity authorized.
- Post-deployment smoke test.
- Deployment evidence recorded.

57. Release Gate

Requirement	Status required
All CI gates	PASS
Safety regression suite	PASS
Security scanning	PASS
Terraform validation	PASS
Staging smoke tests	PASS
Evidence/audit integrity	PASS
Release metadata	COMPLETE
Rollback procedure	VALIDATED

58. Acceptance Criteria

- Repository cleanly separates domain, infrastructure, LLM, evidence, and experiment concerns.
- Dependencies and toolchain versions are pinned.
- Core development does not require AWS access.
- Terraform is versioned, reviewed, and validated through automation.
- CI blocks safety-critical failures.
- CD produces immutable artifacts.
- Secrets are managed outside source control.
- Deployment environments are isolated.
- Rollback and emergency-stop mechanisms exist.
- Trusted State cannot be modified by deployment tooling.
- Every release is traceable to code/configuration versions.
- Developer and safety-critical Definition of Done are enforceable.

59. Non-Goals

- Building a full production SRE platform before MVP.
- Supporting every AWS service.
- Automating unrestricted production deployment.
- Using CI as a substitute for DAIL verification.
- Treating Terraform state as DAIL Trusted State.
- Optimizing infrastructure cost at the expense of safety controls.

60. Final Development & DevOps Directive

DAIL shall be implemented as a reproducible, versioned, fail-closed software system in which deterministic safety logic is isolated from probabilistic generation, infrastructure is managed as code, evidence is preserved, and every release passes automated quality gates.

Development and deployment tooling must remain subordinate to DAIL's trust model: CI/CD may build and deploy the system, but it cannot declare infrastructure trusted. Only the Verification and Promotion pipeline defined in Documents 06–11 can advance DAIL Trusted State.

Appendix A — Developer Daily Flow

```
pull latest
  ↓
create feature branch
  ↓
implement small change
  ↓
run targeted tests
  ↓
run safety regression suite
  ↓
run formatting/lint/static checks
  ↓
commit
  ↓
open PR
  ↓
CI
  ↓
review
  ↓
merge
```

Appendix B — Initial Repository Bootstrap Checklist

- Create repository.
- Create package/module structure.
- Pin runtime/dependencies.
- Configure formatter/linter/type checks.
- Configure pytest/test discovery.
- Create fixture directories.
- Create Terraform structure.
- Create local development environment.
- Create CI workflow.
- Create evidence schemas.
- Create initial database schema/migration.
- Create application health checks.
- Create experiment harness skeleton.
- Create architecture/spec documentation index.

Appendix C — Next Specification

Document 15 — Implementation Roadmap & Definition of Done is the final pre-implementation specification. It converts Documents 01–14 into a sequenced build plan with implementation phases, modules, dependencies, milestones, deliverables, verification gates, demo checkpoints, and explicit completion criteria.